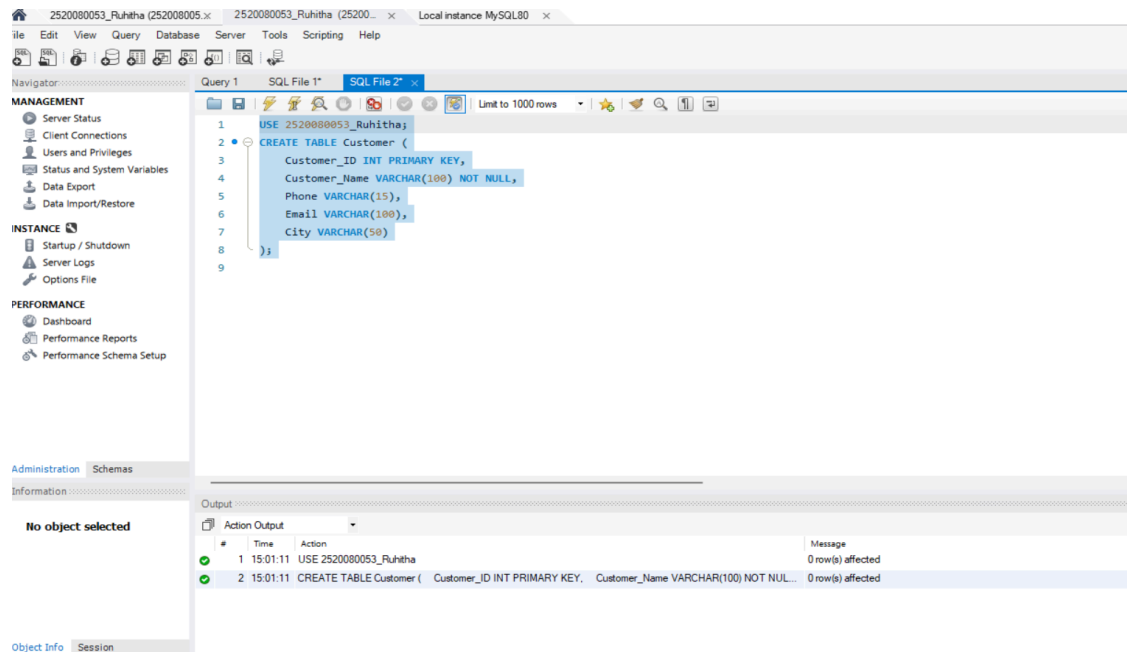
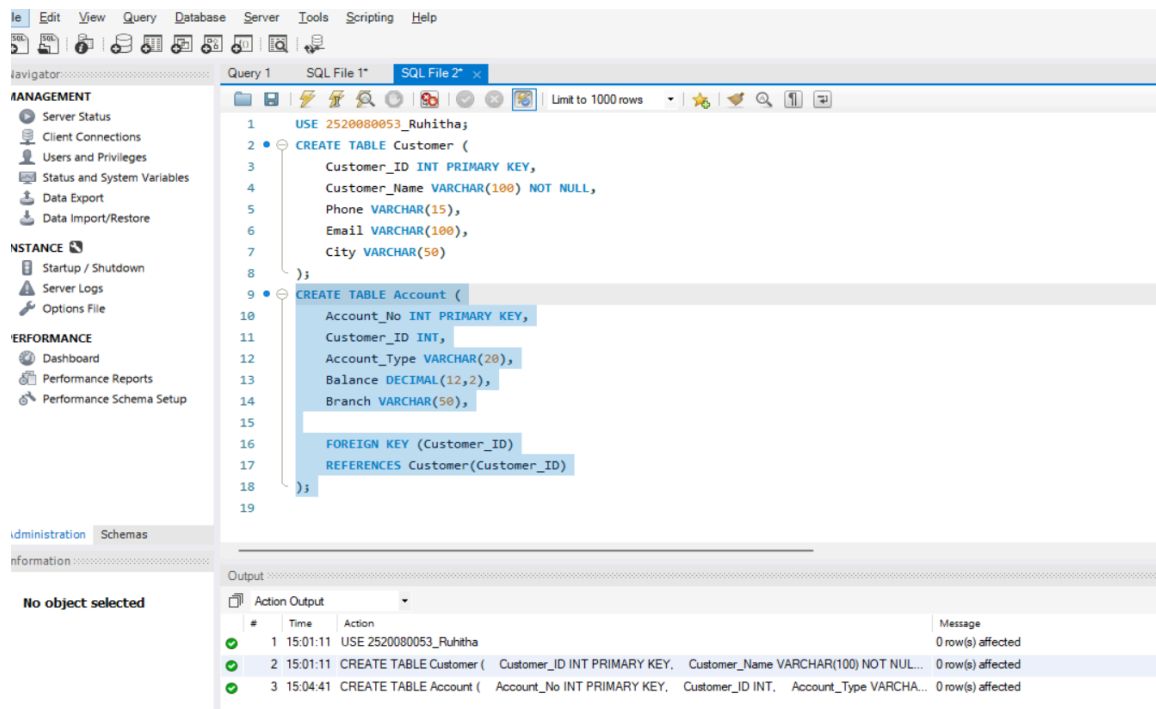


SKILL WEEK 3

1) CREATE CUSTOMER TABLE



2) CREATE ACCOUNT TABLE



3) CREATE BANK TRANSACTION TABLE

2520080053_Ruhitha (252008005.x) 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 SQL File 1* SQL File 2* x Limit to 1000 rows

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

```

12 Account_Type VARCHAR(20),
13 Balance DECIMAL(12,2),
14 Branch VARCHAR(50),
15
16 FOREIGN KEY (Customer_ID)
17 REFERENCES Customer(Customer_ID)
18 );
19 CREATE TABLE Bank_Transaction (
20 Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
21 Account_No INT,
22 Transaction_Type VARCHAR(20),
23 Amount DECIMAL(12,2),
24 Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
25
26 FOREIGN KEY (Account_No)
27 REFERENCES Account(Account_No)
28 );
29
30
31

```

Output

#	Time	Action	Message
1	15:01:11	USE 2520080053_Ruhitha	0 row(s) affected
2	15:01:11	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NULL, ...	0 row(s) affected
3	15:04:41	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARCHAR...	0 row(s) affected
4	15:06:02	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_N...	0 row(s) affected

Object Info Session

4) CREATE LOAN TABLE

2520080053_Ruhitha (252008005.x) 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 SQL File 1* SQL File 2* x Limit to 1000 rows

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

```

21 Account_No INT,
22 Transaction_Type VARCHAR(20),
23 Amount DECIMAL(12,2),
24 Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
25
26 FOREIGN KEY (Account_No)
27 REFERENCES Account(Account_No)
28 );
29 CREATE TABLE Loan (
30 Loan_ID INT PRIMARY KEY,
31 Customer_ID INT,
32 Loan_Type VARCHAR(30),
33 Loan_Amount DECIMAL(12,2),
34 Interest_Rate DECIMAL(5,2),
35
36 FOREIGN KEY (Customer_ID)
37 REFERENCES Customer(Customer_ID)
38 );
39
40

```

Output

#	Time	Action	Message
1	15:01:11	USE 2520080053_Ruhitha	0 row(s) affected
2	15:01:11	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NULL, ...	0 row(s) affected
3	15:04:41	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARCHAR...	0 row(s) affected
4	15:06:02	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_N...	0 row(s) affected
5	15:07:03	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(30), L...	0 row(s) affected

Object Info Session

5) INSERT CUSTOMER DATA

The screenshot shows the MySQL Workbench interface with the following SQL queries in the editor:

```
55 'kiran@gmail.com', 'Hyderabad'),
56
57 (106, 'Anil Kumar', '9876543215',
58 'anil@gmail.com', 'Delhi'),
59
60 (107, 'Meena Reddy', '9876543216',
61 'meena@gmail.com', 'Mumbai'),
62
63 (108, 'Rahul Sharma', '9876543217',
64 'rahul@gmail.com', 'Pune'),
65
66 (109, 'Lakshmi Devi', '9876543218',
67 'lakshmi@gmail.com', 'Hyderabad'),
68
69 (110, 'Suresh Babu', '9876543219',
70 'suresh@gmail.com', 'Vijayawada');
71
72
73
74
75
```

The Output tab shows the following results:

#	Time	Action	Message
2	15:01:11	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NU...	0 row(s) affected
3	15:04:41	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARC...	0 row(s) affected
4	15:06:02	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_...	0 row(s) affected
5	15:07:03	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(30), ...	0 row(s) affected
6	15:08:53	INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES (101, 'Ravi Kumar', '9876...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

6) INSERT ACCOUNT DATA

The screenshot shows the MySQL Workbench interface with the following SQL queries in the editor:

```
70 'suresh@gmail.com', 'Vijayawada');
71
72 INSERT INTO Account
73 (Account_No, Customer_ID, Account_Type, Balance, Branch)
74 VALUES
75 (10001, 101, 'Savings', 50000, 'Hyderabad'),
76 (10002, 102, 'Savings', 75000, 'Vijayawada'),
77 (10003, 103, 'Current', 120000, 'Bangalore'),
78 (10004, 104, 'Savings', 45000, 'Chennai'),
79 (10005, 105, 'Current', 90000, 'Hyderabad'),
80 (10006, 106, 'Savings', 65000, 'Delhi'),
81 (10007, 107, 'Current', 150000, 'Mumbai'),
82 (10008, 108, 'Savings', 35000, 'Pune'),
83 (10009, 109, 'Savings', 85000, 'Hyderabad'),
84 (10010, 110, 'Current', 110000, 'Vijayawada');
85
86
87
88
89
90
```

The Output tab shows the following results:

#	Time	Action	Message
5	15:07:03	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(30), ...	0 row(s) affected
6	15:08:53	INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES (101, 'Ravi Kumar', '987...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
7	15:11:55	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10...	Error Code: 1452. Cannot add or update a child row: a foreign key constrai
8	15:14:17	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	0 row(s) returned
9	15:17:03	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, '...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

7)INSERT TRANSACTION DATA

The screenshot shows the SQL Enterprise Manager interface. The left pane displays the 'MANAGEMENT' tree with 'Server Status' selected. The main pane shows a query window with the following SQL code:

```
84 (10010, 110, 'Current', 110000, 'Vijayawada');
85
86 INSERT INTO Bank_Transaction
87 (Account_No, Transaction_Type, Amount)
88 VALUES
89 (10001, 'DEPOSIT', 10000),
90 (10001, 'WITHDRAW', 5000),
91 (10002, 'DEPOSIT', 15000),
92 (10003, 'WITHDRAW', 20000),
93 (10004, 'DEPOSIT', 5000),
94 (10005, 'WITHDRAW', 10000),
95 (10006, 'DEPOSIT', 12000),
96 (10007, 'DEPOSIT', 25000),
97 (10008, 'WITHDRAW', 5000),
98 (10009, 'DEPOSIT', 20000),
99 (10010, 'WITHDRAW', 15000);
```

The 'Output' pane shows the execution results:

#	Time	Action	Message
6	15:08:53	INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES (101, 'Ravi Kumar', '987...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
7	15:11:55	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails
8	15:14:17	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	0 row(s) returned
9	15:17:03	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, '...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
10	15:18:28	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10...	11 row(s) affected Records: 11 Duplicates: 0 Warnings: 0

8)INSERT LOAN DATA

The screenshot shows the SQL Enterprise Manager interface. The left pane displays the 'MANAGEMENT' tree with 'Server Status' selected. The main pane shows a query window with the following SQL code:

```
97 (10008, 'WITHDRAW', 5000),
98 (10009, 'DEPOSIT', 20000),
99 (10010, 'WITHDRAW', 15000);
100
101 INSERT INTO Loan
102 (Loan_ID, Customer_ID, Loan_Type,
103 Loan_Amount, Interest_Rate)
104 VALUES
105 (501, 101, 'Home Loan', 5000000, 7.5),
106 (502, 102, 'Education Loan', 1000000, 6.5),
107 (503, 103, 'Car Loan', 800000, 8.2),
108 (504, 104, 'Personal Loan', 500000, 10.5),
109 (505, 105, 'Home Loan', 4000000, 7.2),
110 (506, 106, 'Car Loan', 900000, 8.5),
111 (507, 107, 'Business Loan', 3000000, 9.0),
112 (508, 109, 'Personal Loan', 600000, 10.0);
```

The 'Output' pane shows the execution results:

#	Time	Action	Message
7	15:11:55	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10...	Error Code: 1452. Cannot add or update a child row: a
8	15:14:17	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	0 row(s) returned
9	15:17:03	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, '...	10 row(s) affected Records: 10 Duplicates: 0 Warning
10	15:18:28	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10...	11 row(s) affected Records: 11 Duplicates: 0 Warning

9)DISPLAY ORIGINAL TABLES

Customer

The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' section with options like Server Status, Client Connections, Users and Privileges, Status and System Variables, Data Export, and Data Import/Restore. The 'INSTANCE' section includes Startup / Shutdown, Server Logs, and Options File. The 'PERFORMANCE' section includes Dashboard, Performance Reports, and Performance Schema Setup. The main window displays the 'Customer' table data. The table has columns: Customer_ID, Customer_Name, Phone, Email, and City. The data is as follows:

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
106	Anil Kumar	9876543215	anil@gmail.com	Delhi
107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada

The bottom panel shows the 'Output' window with the following actions:

#	Time	Action	Message
8	15:14:17	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	0 row(s) returned
9	15:17:03	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, 'Savings', 50000.00, 'Hyderabad')	10 row(s) affected
10	15:18:28	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10000.00)	11 row(s) affected
11	15:22:08	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES (501, 101, 'Home Loan', 4000000.00, 7.2)	8 row(s) affected
12	15:24:37	SELECT * FROM Customer LIMIT 0, 1000	10 row(s) returned

Account

The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' section with options like Server Status, Client Connections, Users and Privileges, Status and System Variables, Data Export, and Data Import/Restore. The 'INSTANCE' section includes Startup / Shutdown, Server Logs, and Options File. The 'PERFORMANCE' section includes Dashboard, Performance Reports, and Performance Schema Setup. The main window displays the 'Account' table data. The table has columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The data is as follows:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10006	106	Savings	65000.00	Delhi
10007	107	Current	150000.00	Mumbai
10008	108	Savings	35000.00	Pune
10009	109	Savings	85000.00	Hyderabad
10010	110	Current	110000.00	Vijayawada

The bottom panel shows the 'Output' window with the following actions:

#	Time	Action	Message
9	15:17:03	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, 'Savings', 50000.00, 'Hyderabad')	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
10	15:18:28	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10000.00)	11 row(s) affected Records: 11 Duplicates: 0 Warnings: 0
11	15:22:08	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES (501, 101, 'Home Loan', 4000000.00, 7.2)	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0
12	15:24:37	SELECT * FROM Customer LIMIT 0, 1000	10 row(s) returned
13	15:27:11	SELECT * FROM Account LIMIT 0, 1000	10 row(s) returned

Bank_Transaction

The screenshot shows the MySQL Workbench interface. The left sidebar displays the SCHEMAS tree with the database 2520080053_ruhitha selected. The main editor shows a SQL query file with the following content:

```
111 (507, 107, 'Business Loan', 3000000, 9.0),
112 (508, 109, 'Personal Loan', 600000, 10.0);
113
114 • SELECT * FROM Customer;
115 • SELECT * FROM Account;
116 • SELECT * FROM Bank_Transaction;
117
```

The Result Grid shows the data for the Bank_Transaction table:

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
12	10001	DEPOSIT	10000.00	2026-08-12 15:18:28
13	10001	WITHDRAW	5000.00	2026-08-12 15:18:28
14	10002	DEPOSIT	15000.00	2026-08-12 15:18:28
15	10003	WITHDRAW	20000.00	2026-08-12 15:18:28
16	10004	DEPOSIT	5000.00	2026-08-12 15:18:28
17	10005	WITHDRAW	10000.00	2026-08-12 15:18:28
18	10006	DEPOSIT	12000.00	2026-08-12 15:18:28
19	10007	DEPOSIT	25000.00	2026-08-12 15:18:28
20	10008	WITHDRAW	5000.00	2026-08-12 15:18:28
21	10009	DEPOSIT	20000.00	2026-08-12 15:18:28
22	10010	WITHDRAW	15000.00	2026-08-12 15:18:28
•	NULL	NULL	NULL	NULL

The bottom panel shows the Action Output for the query:

#	Time	Action	Message
✓ 10	15:18:28	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10...	11 row(s) affected R
✓ 11	15:22:08	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES (501, 101, '...	8 row(s) affected Re
✓ 12	15:24:37	SELECT * FROM Customer LIMIT 0, 1000	10 row(s) returned
✓ 13	15:27:11	SELECT * FROM Account LIMIT 0, 1000	10 row(s) returned
✓ 14	15:28:13	SELECT * FROM Bank_Transaction LIMIT 0, 1000	11 row(s) returned

Loan

The screenshot shows the MySQL Workbench interface. The left sidebar displays the SCHEMAS tree with the database 2520080053_ruhitha selected. The main editor shows a SQL query file with the following content:

```
111 (507, 107, 'Business Loan', 3000000, 9.0),
112 (508, 109, 'Personal Loan', 600000, 10.0);
113
114 • SELECT * FROM Customer;
115 • SELECT * FROM Account;
116 • SELECT * FROM Bank_Transaction;
117 • SELECT * FROM Loan;
```

The Result Grid shows the data for the Loan table:

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
501	101	Home Loan	5000000.00	7.50
502	102	Education Loan	1000000.00	6.50
503	103	Car Loan	800000.00	8.20
504	104	Personal Loan	500000.00	10.50
505	105	Home Loan	4000000.00	7.20
506	106	Car Loan	900000.00	8.50
507	107	Business Loan	3000000.00	9.00
508	109	Personal Loan	600000.00	10.00
•	NULL	NULL	NULL	NULL

The bottom panel shows the Action Output for the query:

#	Time	Action	Message
✓ 11	15:22:08	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES (501, 101, '...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0
✓ 12	15:24:37	SELECT * FROM Customer LIMIT 0, 1000	10 row(s) returned
✓ 13	15:27:11	SELECT * FROM Account LIMIT 0, 1000	10 row(s) returned
✓ 14	15:28:13	SELECT * FROM Bank_Transaction LIMIT 0, 1000	11 row(s) returned
✓ 15	15:33:10	SELECT * FROM Loan LIMIT 0, 1000	8 row(s) returned

Query Completed

CREATE A VIEW FOR ALL CUSTOMERS

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Schemas list includes 2520080053_ruhitha, Tables, Views, Stored Procedures, Functions, bookflow_db, kludb, and sys.
- Query Editor:** Contains SQL code for creating a view and selecting from it.

```
117 • SELECT * FROM Loan;
118
119 • CREATE VIEW Customer_View AS
120 SELECT *
121 FROM Customer;
122
123 • SELECT * FROM Customer_View;
```
- Result Grid:** Displays a table of customer data with columns: Customer_ID, Customer_Name, Phone, Email, and City. It contains 10 rows of data.
- Output Window:** Shows the execution results of the SQL statements, including error messages for duplicate view creation attempts.

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
106	Anil Kumar	9876543215	anil@gmail.com	Delhi
107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada

CREATE A VIEW FOR CUSTOMER BASIC INFORMATION

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Schemas list includes 2520080053_ruhitha, Tables, Views, Stored Procedures, Functions, bookflow_db, kludb, and sys.
- Query Editor:** Contains SQL code for creating a view and selecting from it.

```
124 • CREATE VIEW Customer_Basic_View AS
125 SELECT
126     Customer_ID,
127     Customer_Name,
128     City
129 FROM Customer;
130 • SELECT * FROM Customer_Basic_View;
```
- Result Grid:** Displays a table of customer basic information with columns: Customer_ID, Customer_Name, and City. It contains 10 rows of data.
- Output Window:** Shows the execution results of the SQL statements, including error messages for duplicate view creation attempts.

Customer_ID	Customer_Name	City
101	Ravi Kumar	Hyderabad
102	Priya Sharma	Vijayawada
103	Arjun Reddy	Bangalore
104	Sneha Rao	Chennai
105	Kiran Kumar	Hyderabad
106	Anil Kumar	Delhi
107	Meena Reddy	Mumbai
108	Rahul Sharma	Pune
109	Lakshmi Devi	Hyderabad
110	Suresh Babu	Vijayawada

CREATE A VIEW FOR ACCOUNT INFORMATION

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows the schema `2520080053_ruhitha` with objects: Tables, Views, Stored Procedures, Functions, `bookflow_db`, `kludb`, and `sys`.
- Query Editor:** Contains the following SQL code:

```
127 Customer_Name,  
128 City  
129 FROM Customer;  
130 • SELECT * FROM Customer_Basic_View;  
131  
132  
133 • CREATE VIEW Account_View AS
```
- Result Grid:** Displays the following data:

Account_No	Account_Type	Balance	Branch
10001	Savings	50000.00	Hyderabad
10002	Savings	75000.00	Vijayawada
10003	Current	120000.00	Bangalore
10004	Savings	45000.00	Chennai
10005	Current	90000.00	Hyderabad
10006	Savings	65000.00	Delhi
10007	Current	150000.00	Mumbai
10008	Savings	35000.00	Pune
10009	Savings	85000.00	Hyderabad
10010	Current	110000.00	Vijayawada
- Output:** Shows the execution log with the following messages:

#	Time	Action	Message
20	15:35:57	SELECT * FROM Customer_View LIMIT 0, 1000	10 row(s) returned
21	15:36:47	CREATE VIEW Customer_Basic_View AS SELECT Customer_ID, Customer_Name, City FROM Customer	0 row(s) affected
22	15:37:01	SELECT * FROM Customer_Basic_View LIMIT 0, 1000	10 row(s) returned
23	15:37:51	CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM ...	0 row(s) affected
24	15:37:51	SELECT * FROM Account_View LIMIT 0, 1000	10 row(s) returned

CREATE A VIEW FOR SAVINGS ACCOUNTS

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows the schema `2520080053_ruhitha` with objects: Tables, Views, Stored Procedures, Functions, `bookflow_db`, `kludb`, and `sys`.
- Query Editor:** Contains the following SQL code:

```
131  
132  
133 • CREATE VIEW Savings_Account_View AS  
134 SELECT *  
135 FROM Account  
136 WHERE Account_Type = 'Savings';  
137 • SELECT * FROM Savings_Account_View;
```
- Result Grid:** Displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10004	104	Savings	45000.00	Chennai
10006	106	Savings	65000.00	Delhi
10008	108	Savings	35000.00	Pune
10009	109	Savings	85000.00	Hyderabad
- Output:** Shows the execution log with the following messages:

#	Time	Action	Message
22	15:37:01	SELECT * FROM Customer_Basic_View LIMIT 0, 1000	10 row(s) returned
23	15:37:51	CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM ...	0 row(s) affected
24	15:37:51	SELECT * FROM Account_View LIMIT 0, 1000	10 row(s) returned
25	15:38:31	CREATE VIEW Savings_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Savings'	0 row(s) affected
26	15:38:31	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 row(s) returned

CREATE A VIEW FOR CURRENT ACCOUNTS

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script with the following queries:

```
137 • SELECT * FROM Savings_Account_View;
138
139 • CREATE VIEW Current_Account_View AS
140 SELECT *
141 FROM Account
142 WHERE Account_Type = 'Current';
143 • SELECT * FROM Current_Account_View;
```

The 'Result Grid' shows the output of the last query, displaying 4 rows of current accounts:

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore
10005	105	Current	90000.00	Hyderabad
10007	107	Current	150000.00	Mumbai
10010	110	Current	110000.00	Vijayawada

The bottom panel shows the 'Action Output' for the current session, listing the execution of the queries and the number of rows returned or affected.

#	Time	Action	Message
24	15:37:51	SELECT * FROM Account_View LIMIT 0, 1000	10 row(s) returned
25	15:38:31	CREATE VIEW Savings_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Savings'	0 row(s) affected
26	15:38:31	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 row(s) returned
27	15:39:24	CREATE VIEW Current_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Current'	0 row(s) affected
28	15:39:25	SELECT * FROM Current_Account_View LIMIT 0, 1000	4 row(s) returned

HIGH BALANCE ACCOUNT VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script with the following queries:

```
148 Customer_ID,
149 Account_Type,
150 Balance
151 FROM Account
152 WHERE Balance > 100000;
153 • SELECT * FROM High_Balance_View;
154
```

The 'Result Grid' shows the output of the last query, displaying 3 rows of high balance accounts:

Account_No	Customer_ID	Account_Type	Balance
10003	103	Current	120000.00
10007	107	Current	150000.00
10010	110	Current	110000.00

The bottom panel shows the 'Action Output' for the current session, listing the execution of the queries and the number of rows returned or affected.

#	Time	Action	Message
26	15:38:31	SELECT * FROM Savings_Account_View LIMIT 0, 1000	6 row(s) returned
27	15:39:24	CREATE VIEW Current_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Current'	0 row(s) affected
28	15:39:25	SELECT * FROM Current_Account_View LIMIT 0, 1000	4 row(s) returned
29	15:40:20	CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan...	0 row(s) affected
30	15:40:20	SELECT * FROM High_Balance_View LIMIT 0, 1000	3 row(s) returned

HYDERABAD ACCOUNT VIEW

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree for the '2520080053_ruhitha' database, including Tables, Views, Stored Procedures, Functions, and other databases like bookflow_db, kludb, and sys. The right pane shows the 'Query 1' window with the following SQL script:

```
153 • SELECT * FROM High_Balance_View;
154 • CREATE VIEW Hyderabad_Account_View AS
155 • SELECT *
156 • FROM Account
157 • WHERE Branch = 'Hyderabad';
158 • SELECT * FROM Hyderabad_Account_View;
159
```

Below the script, the 'Result Grid' displays the data for the 'Hyderabad_Account_View'.

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Hyderabad

The bottom pane shows the 'Output' window with the 'Action Output' tab selected, displaying the execution results of the SQL script.

#	Time	Action	Message
28	15:39:25	SELECT * FROM Current_Account_View LIMIT 0, 1000	4 row(s) returned
29	15:40:20	CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan...	0 row(s) affected
30	15:40:20	SELECT * FROM High_Balance_View LIMIT 0, 1000	3 row(s) returned
31	15:41:03	CREATE VIEW Hyderabad_Account_View AS SELECT * FROM Account WHERE Branch = 'Hyderabad'	0 row(s) affected
32	15:41:03	SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000	3 row(s) returned

LOW BALANCE ACCOUNT VIEW

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree for the '2520080053_ruhitha' database. The right pane shows the 'Query 1' window with the following SQL script:

```
157 WHERE Branch = 'Hyderabad';
158 • SELECT * FROM Hyderabad_Account_View;
159 • CREATE VIEW Low_Balance_View AS
160 • SELECT
161 • Account_No,
162 • Customer_ID,
163 • Balance
164 • FROM Account
165 • WHERE Balance < 50000;
166 • SELECT * FROM Low_Balance_View;
```

Below the script, the 'Result Grid' displays the data for the 'Low_Balance_View'.

Account_No	Customer_ID	Balance
10004	104	45000.00
10008	108	35000.00

The bottom pane shows the 'Output' window with the 'Action Output' tab selected, displaying the execution results of the SQL script.

#	Time	Action	Message
30	15:40:20	SELECT * FROM High_Balance_View LIMIT 0, 1000	3 row(s) returned
31	15:41:03	CREATE VIEW Hyderabad_Account_View AS SELECT * FROM Account WHERE Branch = 'Hyderabad'	0 row(s) affected
32	15:41:03	SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000	3 row(s) returned
33	15:41:48	CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account...	0 row(s) affected
34	15:41:48	SELECT * FROM Low_Balance_View LIMIT 0, 1000	2 row(s) returned

CUSTOMER + ACCOUNT VIEW

Query 1 SQL File 1* SQL File 2* x

```
173 A.Account_Type,  
174 A.Balance,  
175 A.Branch  
176 FROM Customer C  
177 JOIN Account A  
178 ON C.Customer_ID = A.Customer_ID;  
179 SELECT * FROM Customer_Account_View;
```

Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada
103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore
104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai
105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai
108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune
109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad
110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada

Customer_Account_View 14 x

Output

#	Time	Action	Message
32	15:41:03	SELECT * FROM Hyderabad_Account_View LIMIT 0, 1000	3 row(s) returned
33	15:41:48	CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account...	0 row(s) affected
34	15:41:48	SELECT * FROM Low_Balance_View LIMIT 0, 1000	2 row(s) returned
35	15:43:34	CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A...	0 row(s) affected
36	15:43:34	SELECT * FROM Customer_Account_View LIMIT 0, 1000	10 row(s) returned

HYDERABAD CUSTOMER ACCOUNT VIEW

Query 1 SQL File 1* SQL File 2* x

```
179 SELECT * FROM Customer_Account_View;  
180 CREATE VIEW Hyderabad_Customer_Accounts AS  
181 SELECT  
182 C.Customer_Name,  
183 C.City,  
184 A.Account_No,  
185 A.Account_Type,  
186 A.Balance  
187 FROM Customer C  
188 JOIN Account A  
189 ON C.Customer_ID = A.Customer_ID  
190 WHERE A.Branch = 'Hyderabad';  
191 SELECT * FROM Hyderabad_Customer_Accounts;
```

Customer_Name	City	Account_No	Account_Type	Balance
Ravi Kumar	Hyderabad	10001	Savings	50000.00
Kiran Kumar	Hyderabad	10005	Current	90000.00
Lakshmi Devi	Hyderabad	10009	Savings	85000.00

Hyderabad_Customer_Accounts... x

Output

#	Time	Action	Message
34	15:41:48	SELECT * FROM Low_Balance_View LIMIT 0, 1000	2 row(s) returned
35	15:43:34	CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A...	0 row(s) affected
36	15:43:34	SELECT * FROM Customer_Account_View LIMIT 0, 1000	10 row(s) returned
37	15:44:47	CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N...	0 row(s) affected

CUSTOMER LOAN VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor shows a SQL query for creating a view. The 'Result Grid' at the bottom displays the output of the queries, including the successful creation of the 'Customer_Loan_View'.

```
Query 1
196 C.Customer_Name,
197 C.City,
198 L.Loan_ID,
199 L.Loan_Type,
200 L.Loan_Amount,
201 L.Interest_Rate
202 FROM Customer C
203 JOIN Loan L
204 ON C.Customer_ID = L.Customer_ID;
205 SELECT * FROM Customer_Loan_View;
```

Customer_ID	Customer_Name	City	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
101	Ravi Kumar	Hyderabad	501	Home Loan	5000000.00	7.50
102	Priya Sharma	Vijayawada	502	Education Loan	1000000.00	6.50
103	Arjun Reddy	Bangalore	503	Car Loan	800000.00	8.20
104	Sneha Rao	Chennai	504	Personal Loan	500000.00	10.50
105	Kiran Kumar	Hyderabad	505	Home Loan	4000000.00	7.20
106	Anil Kumar	Delhi	506	Car Loan	900000.00	8.50
107	Meena Reddy	Mumbai	507	Business Loan	3000000.00	9.00
109	Lakshmi Devi	Hyderabad	508	Personal Loan	600000.00	10.00

Customer_Loan_View 16 x

Output

#	Time	Action	Message
36	15:43:34	SELECT * FROM Customer_Account_View LIMIT 0, 1000	10 row(s) returned
37	15:44:42	CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N...	0 row(s) affected
38	15:44:42	SELECT * FROM Hyderabad_Customer_Accounts LIMIT 0, 1000	3 row(s) returned
39	19:54:11	CREATE VIEW Customer_Loan_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, L.Lo...	0 row(s) affected
40	19:54:11	SELECT * FROM Customer_Loan_View LIMIT 0, 1000	8 row(s) returned

COMPLETE CUSTOMER BANKING VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor shows a SQL query for creating a view. The 'Result Grid' at the bottom displays the output of the queries, including the successful creation of the 'Customer_Banking_View'.

```
Query 1
212 A.Account_No,
213 A.Account_Type,
214 A.Balance,
215 A.Branch,
216 L.Loan_Type,
217 L.Loan_Amount
218 FROM Customer C
219 LEFT JOIN Account A
220 ON C.Customer_ID = A.Customer_ID
221 LEFT JOIN Loan L
222 ON C.Customer_ID = L.Customer_ID;
223 SELECT * FROM Customer_Banking_View;
```

Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch	Loan_Type	Loan_Amount
101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad	Home Loan	5000000.00
102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada	Education Loan	1000000.00
103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore	Car Loan	800000.00
104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai	Personal Loan	500000.00
105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad	Home Loan	4000000.00
106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi	Car Loan	900000.00
107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai	Business Loan	3000000.00
108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune	Personal Loan	600000.00
109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad	Personal Loan	600000.00
110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada	Personal Loan	600000.00

Customer_Banking_View 17 x

Output

#	Time	Action	Message
41	19:55:18	CREATE VIEW Customer_Banking_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.A...	0 row(s) affected
42	19:55:18	SELECT * FROM Customer_Banking_View LIMIT 0, 1000	10 row(s) returned

TOTAL BANK BALANCE

2520080053_Ruhitha (252008005... x 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator SQL File 1* SQL File 2* x Limit to 1000 rows

SCHEMAS

Filter objects

- 2520080053_ruhitha
 - Tables
 - Views
 - Stored Procedures
 - Functions
- bookflow_db
- kludb
- sys

219 LEFT JOIN Account A
 220 ON C.Customer_ID = A.Customer_ID
 221 LEFT JOIN Loan L
 222 ON C.Customer_ID = L.Customer_ID;
 223 • SELECT * FROM Customer_Banking_View;
 224
 225 • CREATE VIEW Total_Bank_Balance AS
 226 SELECT
 227 SUM(Balance) AS Total_Balance
 228 FROM Account;
 229
 230 • SELECT * FROM Total_Bank_Balance;

Result Grid Filter Rows: Exports Wrap Cell Content:

	Total_Balance
▶	825000.00

Administration Schemas

Information

No object selected

Object Info Session

Total_Bank_Balance 18 x

Output

Action Output

#	Time	Action	Message
✓ 43	19:58:02	CREATE VIEW Total_Bank_Balance AS SELECT SUM(Balance) AS Total_Balance FROM Account	0 row(s) affected
✓ 44	19:58:02	SELECT * FROM Total_Bank_Balance LIMIT 0, 1000	1 row(s) returned

AVERAGE ACCOUNT BALANCE

2520080053_Ruhitha (252008005... x 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator SQL File 1* SQL File 2* x Limit to 1000 rows

SCHEMAS

Filter objects

- 2520080053_ruhitha
 - Tables
 - Views
 - Stored Procedures
 - Functions
- bookflow_db
- kludb
- sys

221 LEFT JOIN Loan L
 222 ON C.Customer_ID = L.Customer_ID;
 223 • SELECT * FROM Customer_Banking_View;
 224
 225 • CREATE VIEW Total_Bank_Balance AS
 226 SELECT
 227 SUM(Balance) AS Total_Balance
 228 FROM Account;
 229
 230 • SELECT * FROM Total_Bank_Balance;
 231
 232 • CREATE VIEW Average_Account_Balance AS

Result Grid Filter Rows: Exports Wrap Cell Content:

	Average_Balance
▶	82500.000000

Administration Schemas

Information

No object selected

Object Info Session

Average_Account_Balance 19 x

Output

Action Output

#	Time	Action	Message
✓ 45	19:58:53	CREATE VIEW Average_Account_Balance AS SELECT AVG(Balance) AS Average_Balance FROM Acco...	0 row(s) affected
✓ 46	19:58:53	SELECT * FROM Average_Account_Balance LIMIT 0, 1000	1 row(s) returned

MAXIMUM ACCOUNT BALANCE

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows the following SQL script:

```
231
232 • CREATE VIEW Average_Account_Balance AS
233 SELECT
234     AVG(Balance) AS Average_Balance
235 FROM Account;
236
237 • SELECT * FROM Average_Account_Balance;
238 • CREATE VIEW Maximum_Balance_View AS
239 SELECT
240     MAX(Balance) AS Maximum_Balance
241 FROM Account;
242 • SELECT * FROM Maximum_Balance_View;
```

The 'Result Grid' shows the output of the final query:

Maximum_Balance
150000.00

The bottom status bar shows the execution log:

#	Time	Action	Message
✓ 47	19:59:53	CREATE VIEW Maximum_Balance_View AS SELECT MAX(Balance) AS Maximum_Balance FROM Account	0 row(s) affected
✓ 48	19:59:53	SELECT * FROM Maximum_Balance_View LIMIT 0, 1000	1 row(s) returned

MINIMUM ACCOUNT BALANCE

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows the following SQL script:

```
238 • CREATE VIEW Maximum_Balance_View AS
239 SELECT
240     MAX(Balance) AS Maximum_Balance
241 FROM Account;
242 • SELECT * FROM Maximum_Balance_View;
243
244 • CREATE VIEW Minimum_Balance_View AS
245 SELECT
246     MIN(Balance) AS Minimum_Balance
247 FROM Account;
248 • SELECT * FROM Minimum_Balance_View;
249
```

The 'Result Grid' shows the output of the final query:

Minimum_Balance
35000.00

The bottom status bar shows the execution log:

#	Time	Action	Message
✓ 49	20:00:43	CREATE VIEW Minimum_Balance_View AS SELECT MIN(Balance) AS Minimum_Balance FROM Account	0 row(s) affected
✓ 50	20:00:43	SELECT * FROM Minimum_Balance_View LIMIT 0, 1000	1 row(s) returned

TOTAL NUMBER OF ACCOUNTS

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window contains the following SQL code:

```
245 SELECT
246     MIN(Balance) AS Minimum_Balance
247 FROM Account;
248 • SELECT * FROM Minimum_Balance_View;
249
250 • CREATE VIEW Account_Count_View AS
251 SELECT
252     COUNT(*) AS Total_Accounts
253 FROM Account;
254
255 • SELECT * FROM Account_Count_View;
256
```

Below the editor, the 'Result Grid' shows the output of the last query:

Total_Accounts
10

The bottom panel shows the 'Output' tab with the following messages:

#	Time	Action	Message
51	20:01:56	CREATE VIEW Account_Count_View AS SELECT COUNT(*) AS Total_Accounts FROM Account	0 row(s) affected
52	20:01:56	SELECT * FROM Account_Count_View LIMIT 0, 1000	1 row(s) returned

BRANCH-WISE ACCOUNT COUNT

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window contains the following SQL code:

```
253 FROM Account;
254
255 • SELECT * FROM Account_Count_View;
256
257
258 • CREATE VIEW Branch_Account_Count AS
259 SELECT
260     Branch,
261     COUNT(*) AS Number_of_Accounts
262 FROM Account
263 GROUP BY Branch;
264 • SELECT * FROM Branch_Account_Count;
```

Below the editor, the 'Result Grid' shows the output of the last query:

Branch	Number_of_Accounts
Hyderabad	3
Vijayawada	2
Bangalore	1
Chennai	1
Delhi	1
Mumbai	1
Pune	1

The bottom panel shows the 'Output' tab with the following messages:

#	Time	Action	Message
53	20:02:59	CREATE VIEW Branch_Account_Count AS SELECT Branch, COUNT(*) AS Number_of_Accounts FROM Account GROUP BY Branch	0 row(s) affected
54	20:02:59	SELECT * FROM Branch_Account_Count LIMIT 0, 1000	7 row(s) returned

BRANCH-WISE TOTAL BALANCE

2520080053_Ruhitha (252008005... x 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS Filter objects 2520080053_ruhitha Tables Views Stored Procedures Functions bookflow_db kludb sys

Query 1 SQL File 1* SQL File 2* x Limit to 1000 rows

```

269 SUM(Balance) AS Total_Balance
270 FROM Account
271 GROUP BY Branch;
272
273 CREATE VIEW Branch_Total_Balance AS
274 SELECT
275     Branch,
276     SUM(Balance) AS Total_Balance
277 FROM Account
278 GROUP BY Branch;
279 SELECT * FROM Branch_Total_Balance;
280

```

Result Grid Filter Rows: Export: Wrap Cell Content:

Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	185000.00
Bangalore	120000.00
Chennai	45000.00
Delhi	65000.00
Mumbai	150000.00
Pune	35000.00

Administration Schemas Information

No object selected

Object Info Session

Branch_Total_Balance 24 x

Output Action Output

#	Time	Action	Message
55	20:06:03	CREATE VIEW Branch_Total_Balance AS SELECT Branch, SUM(Balance) AS Total_Balance FROM Account	0 row(s)
56	20:06:03	SELECT * FROM Branch_Total_Balance LIMIT 0, 1000	7 row(s)

ACCOUNT TYPE COUNT

2520080053_Ruhitha (252008005... x 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS Filter objects 2520080053_ruhitha Tables Views Stored Procedures Functions bookflow_db kludb sys

Query 1 SQL File 1* SQL File 2* x Limit to 1000 rows

```

277 FROM Account
278 GROUP BY Branch;
279 SELECT * FROM Branch_Total_Balance;
280
281 CREATE VIEW Account_Type_Count AS
282 SELECT
283     Account_Type,
284     COUNT(*) AS Number_of_Accounts
285 FROM Account
286 GROUP BY Account_Type;
287 SELECT * FROM Account_Type_Count;
288

```

Result Grid Filter Rows: Export: Wrap Cell Content:

Account_Type	Number_of_Accounts
Savings	6
Current	4

Administration Schemas Information

No object selected

Object Info Session

Account_Type_Count 25 x

Output Action Output

#	Time	Action	Message
57	20:08:22	CREATE VIEW Account_Type_Count AS SELECT Account_Type, COUNT(*) AS Number_of_Accounts	0 row(s) affected
58	20:08:22	SELECT * FROM Account_Type_Count LIMIT 0, 1000	2 row(s) returned

Query Completed

ACCOUNT TYPE TOTAL BALANCE

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script for creating a view named 'Account_Type_Balance'. The script includes a SELECT statement from the 'Account' table, grouped by 'Account_Type', and a SUM function to calculate the total balance. The view is created as 'Account_Type_Balance AS SELECT Account_Type, SUM(Balance) AS Total_Balance FROM Account GROUP BY Account_Type;'. The 'Result Grid' shows the output of the view, displaying two rows: 'Savings' with a total balance of 355000.00 and 'Current' with a total balance of 470000.00. The 'Output' pane at the bottom shows the execution log, indicating that the view was created successfully and the SELECT statement returned 2 rows.

```
285 FROM Account
286 GROUP BY Account_Type;
287 SELECT * FROM Account_Type_Count;
288
289 CREATE VIEW Account_Type_Balance AS
290 SELECT
291     Account_Type,
292     SUM(Balance) AS Total_Balance
293 FROM Account
294 GROUP BY Account_Type;
295 SELECT * FROM Account_Type_Balance;
296
```

Account_Type	Total_Balance
Savings	355000.00
Current	470000.00

Account_Type_Balance 26 x

Output

Action Output

#	Time	Action	Message
59	20:12:10	CREATE VIEW Account_Type_Balance AS SELECT Account_Type, SUM(Balance) AS Total_Balance ...	0 row(s) affected
60	20:12:10	SELECT * FROM Account_Type_Balance LIMIT 0, 1000	2 row(s) returned

BRANCHES WITH MORE THAN ONE ACCOUNT

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script for creating a view named 'Multiple_Account_Branches'. The script includes a SELECT statement from the 'Account' table, grouped by 'Branch', and a COUNT function to calculate the number of accounts per branch. The view is created as 'Multiple_Account_Branches AS SELECT Branch, COUNT(*) AS Number_of_Accounts FROM Account GROUP BY Branch HAVING COUNT(*) > 1;'. The 'Result Grid' shows the output of the view, displaying two rows: 'Hyderabad' with 3 accounts and 'Vijayawada' with 2 accounts. The 'Output' pane at the bottom shows the execution log, indicating that the view was created successfully and the SELECT statement returned 2 rows.

```
293 FROM Account
294 GROUP BY Account_Type;
295 SELECT * FROM Account_Type_Balance;
296
297 CREATE VIEW Multiple_Account_Branches AS
298 SELECT
299     Branch,
300     COUNT(*) AS Number_of_Accounts
301 FROM Account
302 GROUP BY Branch
303 HAVING COUNT(*) > 1;
304 SELECT * FROM Multiple_Account_Branches;
```

Branch	Number_of_Accounts
Hyderabad	3
Vijayawada	2

Multiple_Account_Branches 27 x

Output

Action Output

#	Time	Action	Message
61	20:13:19	CREATE VIEW Multiple_Account_Branches AS SELECT Branch, COUNT(*) AS Number_of_Accounts F...	0 row(s) affected
62	20:13:20	SELECT * FROM Multiple_Account_Branches LIMIT 0, 1000	2 row(s) returned

BRANCHES WITH TOTAL BALANCE ABOVE 100000

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows the following SQL code:

```
303 HAVING COUNT(*) > 1;
304 SELECT * FROM Multiple_Account_Branches;
305
306 CREATE VIEW Rich_Branches AS
307 SELECT
308     Branch,
309     SUM(Balance) AS Total_Balance
310 FROM Account
311 GROUP BY Branch
312 HAVING SUM(Balance) > 100000;
313 SELECT * FROM Rich_Branches;
314
```

The 'Result Grid' shows the output of the query:

Branch	Total_Balance
Hyderabad	225000.00
Vijayawada	185000.00
Bangalore	120000.00
Mumbai	150000.00

The 'Output' pane at the bottom shows the execution log:

#	Time	Action	Message
63	20:16:19	CREATE VIEW Rich_Branches AS SELECT Branch, SUM(Balance) AS Total_Balance FROM Account ...	0 row(s) affected
64	20:16:19	SELECT * FROM Rich_Branches LIMIT 0, 1000	4 row(s) returned

DEPOSIT TRANSACTION VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows the following SQL code:

```
310 FROM Account
311 GROUP BY Branch
312 HAVING SUM(Balance) > 100000;
313 SELECT * FROM Rich_Branches;
314
315
316 CREATE VIEW Deposit_Transaction_View AS
317 SELECT *
318 FROM Bank_Transaction
319 WHERE Transaction_Type = 'DEPOSIT';
320 SELECT * FROM Deposit_Transaction_View;
321
```

The 'Result Grid' shows the output of the query:

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
12	10001	DEPOSIT	10000.00	2026-08-12 15:18:28
14	10002	DEPOSIT	15000.00	2026-08-12 15:18:28
16	10004	DEPOSIT	5000.00	2026-08-12 15:18:28
18	10006	DEPOSIT	12000.00	2026-08-12 15:18:28
19	10007	DEPOSIT	25000.00	2026-08-12 15:18:28
21	10009	DEPOSIT	20000.00	2026-08-12 15:18:28

The 'Output' pane at the bottom shows the execution log:

#	Time	Action	Message
65	20:18:30	CREATE VIEW Deposit_Transaction_View AS SELECT * FROM Bank_Transaction WHERE Transaction_Ty...	0 row(s) affected
66	20:18:30	SELECT * FROM Deposit_Transaction_View LIMIT 0, 1000	6 row(s) returned

WITHDRAWAL TRANSACTION VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor shows a SQL script for creating a view. The 'Result Grid' displays the results of the view, showing a list of withdrawal transactions.

```
316 CREATE VIEW Deposit_Transaction_View AS
317 SELECT *
318 FROM Bank_Transaction
319 WHERE Transaction_Type = 'DEPOSIT';
320 SELECT * FROM Deposit_Transaction_View;
321
322 CREATE VIEW Withdrawal_Transaction_View AS
323 SELECT *
324 FROM Bank_Transaction
325 WHERE Transaction_Type = 'WITHDRAW';
326 SELECT * FROM Withdrawal_Transaction_View;
327
```

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
13	10001	WITHDRAW	5000.00	2026-08-12 15:18:28
15	10003	WITHDRAW	20000.00	2026-08-12 15:18:28
17	10005	WITHDRAW	10000.00	2026-08-12 15:18:28
20	10008	WITHDRAW	5000.00	2026-08-12 15:18:28
22	10010	WITHDRAW	15000.00	2026-08-12 15:18:28

Output:

#	Time	Action	Message
67	20:19:28	CREATE VIEW Withdrawal_Transaction_View AS SELECT * FROM Bank_Transaction WHERE Transaction_...	0 row(s) affected
68	20:19:29	SELECT * FROM Withdrawal_Transaction_View LIMIT 0, 1000	5 row(s) returned

TRANSACTION DETAILS WITH CUSTOMER

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor shows a SQL script for creating a view. The 'Result Grid' displays the results of the view, showing a list of transactions with customer details.

```
331 A.Account_No,
332 A.Account_Type,
333 T.Transaction_ID,
334 T.Transaction_Type,
335 T.Amount,
336 T.Transaction_Date
337 FROM Customer C
338 JOIN Account A
339 ON C.Customer_ID = A.Customer_ID
340 JOIN Bank_Transaction T
341 ON A.Account_No = T.Account_No;
342 SELECT * FROM Customer_Transaction_View;
```

Customer_Name	Account_No	Account_Type	Transaction_ID	Transaction_Type	Amount	Transaction_Date
Ravi Kumar	10001	Savings	12	DEPOSIT	10000.00	2026-08-12 15:18:28
Ravi Kumar	10001	Savings	13	WITHDRAW	5000.00	2026-08-12 15:18:28
Priya Sharma	10002	Savings	14	DEPOSIT	15000.00	2026-08-12 15:18:28
Arjun Reddy	10003	Current	15	WITHDRAW	20000.00	2026-08-12 15:18:28
Sneha Rao	10004	Savings	16	DEPOSIT	5000.00	2026-08-12 15:18:28
Kiran Kumar	10005	Current	17	WITHDRAW	10000.00	2026-08-12 15:18:28
Anil Kumar	10006	Savings	18	DEPOSIT	12000.00	2026-08-12 15:18:28
Meena Reddy	10007	Current	19	DEPOSIT	25000.00	2026-08-12 15:18:28
Rahul Sharma	10008	Savings	20	WITHDRAW	5000.00	2026-08-12 15:18:28
Lakshmi Devi	10009	Savings	21	DEPOSIT	20000.00	2026-08-12 15:18:28
Suresh Babu	10010	Current	22	WITHDRAW	15000.00	2026-08-12 15:18:28

Output:

#	Time	Action	Message
69	20:21:11	CREATE VIEW Customer_Transaction_View AS SELECT C.Customer_Name, A.Account_No, A.Acco...	0 row(s) affected
70	20:21:11	SELECT * FROM Customer_Transaction_View LIMIT 0, 1000	11 row(s) returned

HIGH VALUE TRANSACTION VIEW

The screenshot shows the SQL Developer interface with a query window open. The query is as follows:

```
337 FROM Customer C
338 JOIN Account A
339 ON C.Customer_ID = A.Customer_ID
340 JOIN Bank_Transaction T
341 ON A.Account_No = T.Account_No;
342 • SELECT * FROM Customer_Transaction_View;
343
344 • CREATE VIEW High_Value_Transaction_View AS
345 SELECT *
346 FROM Bank_Transaction
347 WHERE Amount > 10000;
348 • SELECT * FROM High_Value_Transaction_View;
```

The result grid shows the following data:

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
14	10002	DEPOSIT	15000.00	2026-08-12 15:18:28
15	10003	WITHDRAW	20000.00	2026-08-12 15:18:28
18	10006	DEPOSIT	12000.00	2026-08-12 15:18:28
19	10007	DEPOSIT	25000.00	2026-08-12 15:18:28
21	10009	DEPOSIT	20000.00	2026-08-12 15:18:28
22	10010	WITHDRAW	15000.00	2026-08-12 15:18:28

The output window shows the following messages:

```
71 20:23:49 CREATE VIEW High_Value_Transaction_View AS SELECT * FROM Bank_Transaction WHERE Amount > 10... 0 row(s) affected
72 20:23:49 SELECT * FROM High_Value_Transaction_View LIMIT 0, 1000 6 row(s) returned
```

ACCOUNTS ORDERED BY BALANCE

The screenshot shows the SQL Developer interface with a query window open. The query is as follows:

```
347 WHERE Amount > 10000;
348 • SELECT * FROM High_Value_Transaction_View;
349
350 • CREATE VIEW Balance_Ranking_View AS
351 SELECT
352 Account_No,
353 Customer_ID,
354 Account_Type,
355 Balance
356 FROM Account
357 ORDER BY Balance DESC;
358 • SELECT * FROM Balance_Ranking_View;
```

The result grid shows the following data:

Account_No	Customer_ID	Account_Type	Balance
10007	107	Current	150000.00
10003	103	Current	120000.00
10010	110	Current	110000.00
10005	105	Current	90000.00
10009	109	Savings	85000.00
10002	102	Savings	75000.00
10006	106	Savings	65000.00
10001	101	Savings	50000.00
10004	104	Savings	45000.00
10008	108	Savings	35000.00

The output window shows the following messages:

```
73 20:29:38 CREATE VIEW Balance_Ranking_View AS SELECT Account_No, Customer_ID, Account_Type, B... 0 row(s) affected
74 20:29:38 SELECT * FROM Balance_Ranking_View LIMIT 0, 1000 10 row(s) returned
```

CUSTOMERS ORDERED BY NAME

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script for creating a view named 'Customer_Name_View'. The script is as follows:

```
356 FROM Account
357 ORDER BY Balance DESC;
358 • SELECT * FROM Balance_Ranking_View;
359
360 • CREATE VIEW Customer_Name_View AS
361 SELECT
362     Customer_ID,
363     Customer_Name,
364     City
365 FROM Customer
366 ORDER BY Customer_Name;
367 • SELECT * FROM Customer_Name_View;
```

The 'Result Grid' shows the data returned by the final SELECT statement:

Customer_ID	Customer_Name	City
106	Anil Kumar	Delhi
103	Arjun Reddy	Bangalore
105	Kiran Kumar	Hyderabad
109	Lakshmi Devi	Hyderabad
107	Meena Reddy	Mumbai
102	Priya Sharma	Vijayawada
108	Rahul Sharma	Pune
101	Ravi Kumar	Hyderabad
104	Sneha Rao	Chennai
110	Suresh Babu	Vijayawada

The bottom status bar shows the execution log:

#	Time	Action	Message
75	20:30:26	CREATE VIEW Customer_Name_View AS SELECT Customer_ID, Customer_Name, City FROM Custo...	0 row(s) affected
76	20:30:26	SELECT * FROM Customer_Name_View LIMIT 0, 1000	10 row(s) returned

LOAN INTEREST CALCULATION VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script for creating a view named 'Loan_Interest_View'. The script is as follows:

```
368
369 • CREATE VIEW Loan_Interest_View AS
370 SELECT
371     Loan_ID,
372     Customer_ID,
373     Loan_Type,
374     Loan_Amount,
375     Interest_Rate,
376     (Loan_Amount * Interest_Rate / 100)
377     AS Annual_Interest
378 FROM Loan;
379 • SELECT * FROM Loan_Interest_View;
```

The 'Result Grid' shows the data returned by the final SELECT statement:

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Annual_Interest
501	101	Home Loan	5000000.00	7.50	375000.00000000
502	102	Education Loan	1000000.00	6.50	65000.00000000
503	103	Car Loan	800000.00	8.20	65600.00000000
504	104	Personal Loan	500000.00	10.50	52500.00000000
505	105	Home Loan	4000000.00	7.20	288000.00000000
506	106	Car Loan	900000.00	8.50	76500.00000000
507	107	Business Loan	3000000.00	9.00	270000.00000000
508	109	Personal Loan	600000.00	10.00	60000.00000000

The bottom status bar shows the execution log:

#	Time	Action	Message
77	20:31:15	CREATE VIEW Loan_Interest_View AS SELECT Loan_ID, Customer_ID, Loan_Type, Loan_Amount...	0 row(s) affected
78	20:31:15	SELECT * FROM Loan_Interest_View LIMIT 0, 1000	8 row(s) returned

LOAN TOTAL AMOUNT VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL query in 'Query 1' that creates a view named 'Loan_Total_Amount_View' and then selects all data from it, limited to 1000 rows.

```
382 SELECT
383     Loan_ID,
384     Customer_ID,
385     Loan_Type,
386     Loan_Amount,
387     Interest_Rate,
388     Loan_Amount +
389     (Loan_Amount * Interest_Rate / 100)
390     AS Total_Amount
391 FROM Loan;
392 • SELECT * FROM Loan_Total_Amount_View;
393
```

The 'Result Grid' shows the output of the query, displaying columns: Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate, and Total_Amount. The data includes 8 rows of loan information.

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Total_Amount
501	101	Home Loan	5000000.00	7.50	5375000.00000000
502	102	Education Loan	1000000.00	6.50	1065000.00000000
503	103	Car Loan	800000.00	8.20	865600.00000000
504	104	Personal Loan	500000.00	10.50	552500.00000000
505	105	Home Loan	4000000.00	7.20	4288000.00000000
506	106	Car Loan	900000.00	8.50	976500.00000000
507	107	Business Loan	3000000.00	9.00	3270000.00000000
508	109	Personal Loan	600000.00	10.00	660000.00000000

The 'Output' pane shows the execution log with two entries: a successful 'CREATE VIEW' statement and a successful 'SELECT * FROM Loan_Total_Amount_View LIMIT 0, 1000' statement.

DISPLAY HIGH BALANCE ACCOUNTS FROM VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL query in 'Query 1' that selects all data from the 'Loan_Total_Amount_View' and then filters for accounts with a balance greater than 120,000, limited to 1000 rows.

```
385 Loan_Type,
386 Loan_Amount,
387 Interest_Rate,
388 Loan_Amount +
389 (Loan_Amount * Interest_Rate / 100)
390 AS Total_Amount
391 FROM Loan;
392 • SELECT * FROM Loan_Total_Amount_View;
393
394 • SELECT *
395 FROM High_Balance_View
396 WHERE Balance > 120000;
```

The 'Result Grid' shows the output of the query, displaying columns: Account_No, Customer_ID, Account_Type, and Balance. The data includes 1 row of account information.

Account_No	Customer_ID	Account_Type	Balance
10007	107	Current	150000.00

The 'Output' pane shows the execution log with two entries: a successful 'SELECT * FROM Loan_Total_Amount_View LIMIT 0, 1000' statement and a successful 'SELECT * FROM High_Balance_View WHERE Balance > 120000 LIMIT 0, 1000' statement.

DISPLAY SAVINGS ACCOUNTS ABOVE 60000

Query 1 SQL File 1* SQL File 2* x

```
390 AS Total_Amount
391 FROM Loan;
392 SELECT * FROM Loan_Total_Amount_View;
393
394 SELECT *
395 FROM High_Balance_View
396 WHERE Balance > 120000;
397
398 SELECT *
399 FROM Savings_Account_View
400 WHERE Balance > 60000;
401
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10002	102	Savings	75000.00	Vijayawada
10006	106	Savings	65000.00	Delhi
10009	109	Savings	85000.00	Hyderabad

Savings_Account_View 38 x

Output

Action Output

#	Time	Action	Message
81	20:32:51	SELECT * FROM High_Balance_View WHERE Balance > 120000 LIMIT 0, 1000	1 row(s) returned
82	20:33:36	SELECT * FROM Savings_Account_View WHERE Balance > 60000 LIMIT 0, 1000	3 row(s) returned

DISPLAY HYDERABAD ACCOUNTS ABOVE 50000

Query 1 SQL File 1* SQL File 2* x

```
393
394 SELECT *
395 FROM High_Balance_View
396 WHERE Balance > 120000;
397
398 SELECT *
399 FROM Savings_Account_View
400 WHERE Balance > 60000;
401
402 SELECT *
403 FROM Hyderabad_Account_View
404 WHERE Balance > 50000;
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Hyderabad

Hyderabad_Account_View 39 x

Output

Action Output

#	Time	Action	Message
82	20:33:36	SELECT * FROM Savings_Account_View WHERE Balance > 60000 LIMIT 0, 1000	3 row(s) returned
83	20:38:02	SELECT * FROM Hyderabad_Account_View WHERE Balance > 50000 LIMIT 0, 1000	2 row(s) returned

DISPLAY CUSTOMERS FROM HYDERABAD

The screenshot shows the SQL Developer interface with a query window open. The query is as follows:

```
398 SELECT *
399 FROM Savings_Account_View
400 WHERE Balance > 60000;
401
402 SELECT *
403 FROM Hyderabad_Account_View
404 WHERE Balance > 50000;
405
406 SELECT *
407 FROM Customer_Basic_View
408 WHERE City = 'Hyderabad';
409
```

The result grid shows the following data:

Customer_ID	Customer_Name	City
101	Ravi Kumar	Hyderabad
105	Kiran Kumar	Hyderabad
109	Lakshmi Devi	Hyderabad

The bottom panel shows the execution log with the following messages:

#	Time	Action	Message
83	20:38:02	SELECT * FROM Hyderabad_Account_View WHERE Balance > 50000 LIMIT 0, 1000	2 row(s) returned
84	20:39:04	SELECT * FROM Customer_Basic_View WHERE City = 'Hyderabad' LIMIT 0, 1000	3 row(s) returned

DISPLAY CUSTOMER ACCOUNTS ORDERED BY BALANCE

The screenshot shows the SQL Developer interface with a query window open. The query is as follows:

```
403 FROM Hyderabad_Account_View
404 WHERE Balance > 50000;
405
406 SELECT *
407 FROM Customer_Basic_View
408 WHERE City = 'Hyderabad';
409
410 SELECT *
411 FROM Customer_Account_View
412 ORDER BY Balance DESC;
413
414
```

The result grid shows the following data:

Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai
103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore
110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada
105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad
102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada
106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai
108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune

The bottom panel shows the execution log with the following messages:

#	Time	Action	Message
84	20:39:04	SELECT * FROM Customer_Basic_View WHERE City = 'Hyderabad' LIMIT 0, 1000	3 row(s) returned
85	20:40:12	SELECT * FROM Customer_Account_View ORDER BY Balance DESC LIMIT 0, 1000	10 row(s) returned

DISPLAY LOANS ABOVE 1000000

The screenshot shows the MySQL Workbench interface with the following components:

- Navigator:** Schemas list includes 2520080053_ruhitha, Tables, Views, Stored Procedures, Functions, bookflow_db, kludb, and sys.
- Query 1:** SQL File 1* and SQL File 2* tabs. The SQL code is:

```
405
406 • SELECT *
407 FROM Customer_Basic_View
408 WHERE City = 'Hyderabad';
409
410 • SELECT *
411 FROM Customer_Account_View
412 ORDER BY Balance DESC;
413
414 • SELECT *
415 FROM Customer_Loan_View
416 WHERE Loan_Amount > 1000000;
```
- Result Grid:** Displays the results of the queries. The first query returns 10 rows, and the second query returns 3 rows.
- Output:** Action Output section showing the execution of the queries. The first query (SELECT * FROM Customer_Account_View ORDER BY Balance DESC LIMIT 0, 1000) returned 10 row(s). The second query (SELECT * FROM Customer_Loan_View WHERE Loan_Amount > 1000000 LIMIT 0, 1000) returned 3 row(s).

Customer_ID	Customer_Name	City	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
101	Ravi Kumar	Hyderabad	501	Home Loan	5000000.00	7.50
105	Kiran Kumar	Hyderabad	505	Home Loan	4000000.00	7.20
107	Meena Reddy	Mumbai	507	Business Loan	3000000.00	9.00

UPDATE ACCOUNT BALANCE THROUGH VIEW

The screenshot shows the MySQL Workbench interface with the following components:

- Navigator:** Schemas list includes 2520080053_ruhitha, Tables, Views, Stored Procedures, Functions, bookflow_db, kludb, and sys.
- Query 1:** SQL File 1* and SQL File 2* tabs. The SQL code is:

```
412 ORDER BY Balance DESC;
413
414 • SELECT *
415 FROM Customer_Loan_View
416 WHERE Loan_Amount > 1000000;
417
418 • UPDATE Account_View
419 SET Balance = 60000
420 WHERE Account_No = 10001;
421 • SELECT *
422 FROM Account
423 WHERE Account_No = 10001;
```
- Result Grid:** Displays the results of the queries. The first query returns 1 row, and the second query returns 1 row.
- Output:** Action Output section showing the execution of the queries. The first query (UPDATE Account_View SET Balance = 60000 WHERE Account_No = 10001) affected 1 row(s) and changed 1 row(s). The second query (SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000) returned 1 row(s).

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad

INSERT USING SIMPLE VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL query in 'Query 1' that inserts a new record into the 'Simple_Account_View'.

```
427 Account_No,  
428 Customer_ID,  
429 Account_Type,  
430 Balance,  
431 Branch  
432 FROM Account;  
433 INSERT INTO Simple_Account_View  
434 VALUES  
435 (10011, 101, 'Savings', 55000, 'Hyderabad');  
436 SELECT *  
437 FROM Account;  
438
```

The 'Result Grid' shows the existing data in the 'Account' table:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10006	106	Savings	65000.00	Delhi
10007	107	Current	150000.00	Mumbai
10008	108	Savings	35000.00	Pune
10009	109	Savings	85000.00	Hyderabad
10010	110	Current	110000.00	Vijayawada
10011	101	Savings	55000.00	Hyderabad

The 'Output' pane shows the execution results:

#	Time	Action	Message
90	20:45:02	INSERT INTO Simple_Account_View VALUES (10011, 101, 'Savings', 55000, 'Hyderabad')	1 row(s) affected
91	20:45:02	SELECT * FROM Account LIMIT 0, 1000	11 row(s) returned

DELETE USING SIMPLE VIEW

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL query in 'Query 1' that deletes a record from the 'Simple_Account_View'.

```
432 FROM Account;  
433 INSERT INTO Simple_Account_View  
434 VALUES  
435 (10011, 101, 'Savings', 55000, 'Hyderabad');  
436 SELECT *  
437 FROM Account;  
438  
439 DELETE FROM Simple_Account_View  
440 WHERE Account_No = 10011;  
441 SELECT *  
442 FROM Account;  
443
```

The 'Result Grid' shows the existing data in the 'Account' table:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
10006	106	Savings	65000.00	Delhi
10007	107	Current	150000.00	Mumbai
10008	108	Savings	35000.00	Pune
10009	109	Savings	85000.00	Hyderabad
10010	110	Current	110000.00	Vijayawada

The 'Output' pane shows the execution results:

#	Time	Action	Message
92	20:46:31	DELETE FROM Simple_Account_View WHERE Account_No = 10011	1 row(s) affected
93	20:46:31	SELECT * FROM Account LIMIT 0, 1000	10 row(s) returned

SHOW ALL VIEWS

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with '2520080053_ruhitha' selected. The main query editor contains the following SQL code:

```
436 SELECT *
437 FROM Account;
438
439 DELETE FROM Simple_Account_View
440 WHERE Account_No = 10011;
441 SELECT *
442 FROM Account;
443
444 SHOW FULL TABLES
445 WHERE TABLE_TYPE = 'VIEW';
446
447
```

The 'Result Grid' shows the output of the query:

Tables_in_2520080053_ruhitha	Table_type
account_count_view	VIEW
account_type_balance	VIEW
account_type_count	VIEW
account_view	VIEW
average_account_balance	VIEW
balance_ranking_view	VIEW
branch_account_count	VIEW
branch_total_balance	VIEW
current_account_view	VIEW
customer_account_view	VIEW
customer_banking_view	VIEW

The 'Output' pane shows the execution log:

#	Time	Action	Message
93	20:46:31	SELECT * FROM Account LIMIT 0, 1000	10 row(s) returned
94	20:47:48	SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW'	32 row(s) returned

SHOW VIEW DEFINITION

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with '2520080053_ruhitha' selected. The main query editor contains the following SQL code:

```
437 FROM Account;
438
439 DELETE FROM Simple_Account_View
440 WHERE Account_No = 10011;
441 SELECT *
442 FROM Account;
443
444 SHOW FULL TABLES
445 WHERE TABLE_TYPE = 'VIEW';
446
447 SHOW CREATE VIEW Customer_Account_View;
448
```

The 'Result Grid' shows the output of the query:

View	Create View	character_set_client	collation_connection
customer_account_view	CREATE ALGORITHM=UNDEFINED DEFINER='r...'	utf8mb4	utf8mb4_0900_ai_ci

The 'Output' pane shows the execution log:

#	Time	Action	Message
94	20:47:48	SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW'	32 row(s) returned
95	20:49:52	SHOW CREATE VIEW Customer_Account_View	1 row(s) returned

DESCRIBE VIEW

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows the schema **2520080053_ruhitha** with sub-items: Tables, Views, Stored Procedures, Functions, bookflow_db, kludb, and sys.
- Query Editor:** Contains the following SQL script:

```
438
439 • DELETE FROM Simple_Account_View
440 WHERE Account_No = 10011;
441 • SELECT *
442 FROM Account;
443
444 • SHOW FULL TABLES
445 WHERE TABLE_TYPE = 'VIEW';
446
447 • SHOW CREATE VIEW Customer_Account_View;
448
449 • DESCRIBE Customer_Account_View;
```
- Result Grid:** Displays the structure of the **Customer_Account_View** table.

Field	Type	Null	Key	Default	Extra
Customer_ID	int	NO			
Customer_Name	varchar(100)	NO			
City	varchar(50)	YES			
Account_No	int	NO			
Account_Type	varchar(20)	YES			
Balance	decimal(12,2)	YES			
Branch	varchar(50)	YES			
- Output:** Shows the action output for the DESCRIBE statement.

#	Time	Action	Message
1	20:52:55	DESCRIBE Customer_Account_View	7 row(s) returned

DROP ONE VIEW

The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows the schema **2520080053_ruhitha** with sub-items: Tables, Views, Stored Procedures, Functions, bookflow_db, kludb, and sys.
- Query Editor:** Contains the following SQL script:

```
430 Balance,
431 Branch
432 FROM Account;
433 • INSERT INTO Simple_Account_View
434 VALUES
435 (10011, 101, 'Savings', 55000, 'Hyderabad');
436 • SELECT *
437 FROM Account;
438
439 • DELETE FROM Simple_Account_View
440 WHERE Account_No = 10011;
441 • SELECT *
442 FROM Account;
443
444 • SHOW FULL TABLES
445 WHERE TABLE_TYPE = 'VIEW';
446
447 • SHOW CREATE VIEW Customer_Account_View;
448
449 • DESCRIBE Customer_Account_View;
450 • DROP VIEW Customer_View;
451
452
453
```
- Output:** Shows the action output for the DROP VIEW statement.

#	Time	Action	Message
1	20:52:55	DESCRIBE Customer_Account_View	7 row(s) returned
2	20:57:38	DROP VIEW Customer_View	0 row(s) affected

DROP MULTIPLE VIEWS

2520080053_Ruhitha (2520080053_...)

2520080053_Ruhitha (25200...)

Local instance MySQL80

File

Edit

View

Query

Database

Server

Tools

Scripting

Help

Navigator

Filter objects

2520080053_ruhitha

Tables

Views

Stored Procedures

Functions

bookflow_db

kludb

sys

Administration

Schemas

Information

Schema: 2520080053_ruhitha

Object Info

Session

Query 1

SQL File 1*

SQL File 2*

Limit to 1000 rows

432 FROM Account;

433 INSERT INTO Simple_Account_View

434 VALUES

435 (10011, 101, 'Savings', 55000, 'Hyderabad');

436 SELECT *

437 FROM Account;

438

439 DELETE FROM Simple_Account_View

440 WHERE Account_No = 10011;

441 SELECT *

442 FROM Account;

443

444 SHOW FULL TABLES

445 WHERE TABLE_TYPE = 'VIEW';

446

447 SHOW CREATE VIEW Customer_Account_View;

448

449 DESCRIBE Customer_Account_View;

450 DROP VIEW Customer_View;

451

452 DROP VIEW

453 Customer_Basic_View,

454 Savings_Account_View,

455 Current_Account_View;

456

Output

Action Output

#	Time	Action	Message
2	20:57:38	DROP VIEW Customer_View	0 row(s) affected
3	20:58:18	DROP VIEW Customer_Basic_View, Savings_Account_View, Current_Account_View	0 row(s) affected